Database & Management System

[LAB MANUAL]

K.R. MANGALAM UNIVERSITY

SOET

(School of Engineering & Technology)

B.TECH CSE (CORE)

Submitted by:
Khushi
Roll no: **2401010089**

Submitted to:
Mansi Kajal

Experiment-1

Aim

To design an Entity–Relationship (ER) diagram for a company database system and define the relationship types between entities.

Objectives

- To identify entities and their attributes in a company database
- To represent relationships among entities
- To define relationship types such as one-to-one, one-to-many, and many-to-many

Entities and Attributes

1. Employee

Represents employees working in the company.

Attributes:

- SSN (Primary Key)
- Name
- Address
- Sex
- Salary
- SuperSSN
- DNo

2. Department

Represents departments in the company.

Attributes:

- DNo (Primary Key)
- DName
- MgrSSN
- MgrStartDate

3. Project

Represents projects handled by the company.

Attributes:

- PNo (Primary Key)
- PName

- PLocation
- DNo

4. DLocation

Represents locations of departments.

Attributes:

- DLoc
- DNo

5. Works_On

Represents the association between employees and projects.

Attributes:

- SSN
- PNo
- Hours

<u>Relationships and Relationship Types</u>

1. Works_In

Entities involved: Employee – Department

Relationship type:

- **Many-to-One (N : 1)**

Explanation:
Many employees can work in one department, but each employee works in only one department.

2. Manages

Entities involved: Employee – Department

Relationship type:

- **One-to-One (1 : 1)**

Explanation:
Each department is managed by one employee, and an employee manages at most one department.

3. Has_Location

Entities involved: Department – DLocation

Relationship type:

- **One-to-Many (1 : N)**

Explanation:
A department can have multiple locations, but each location belongs to only one department.

4. Controls / Involves

Entities involved: Department – Project

Relationship type:

- **One-to-Many (1 : N)**

Explanation:
A department can control multiple projects, but each project is controlled by only one department.

5. Works_On

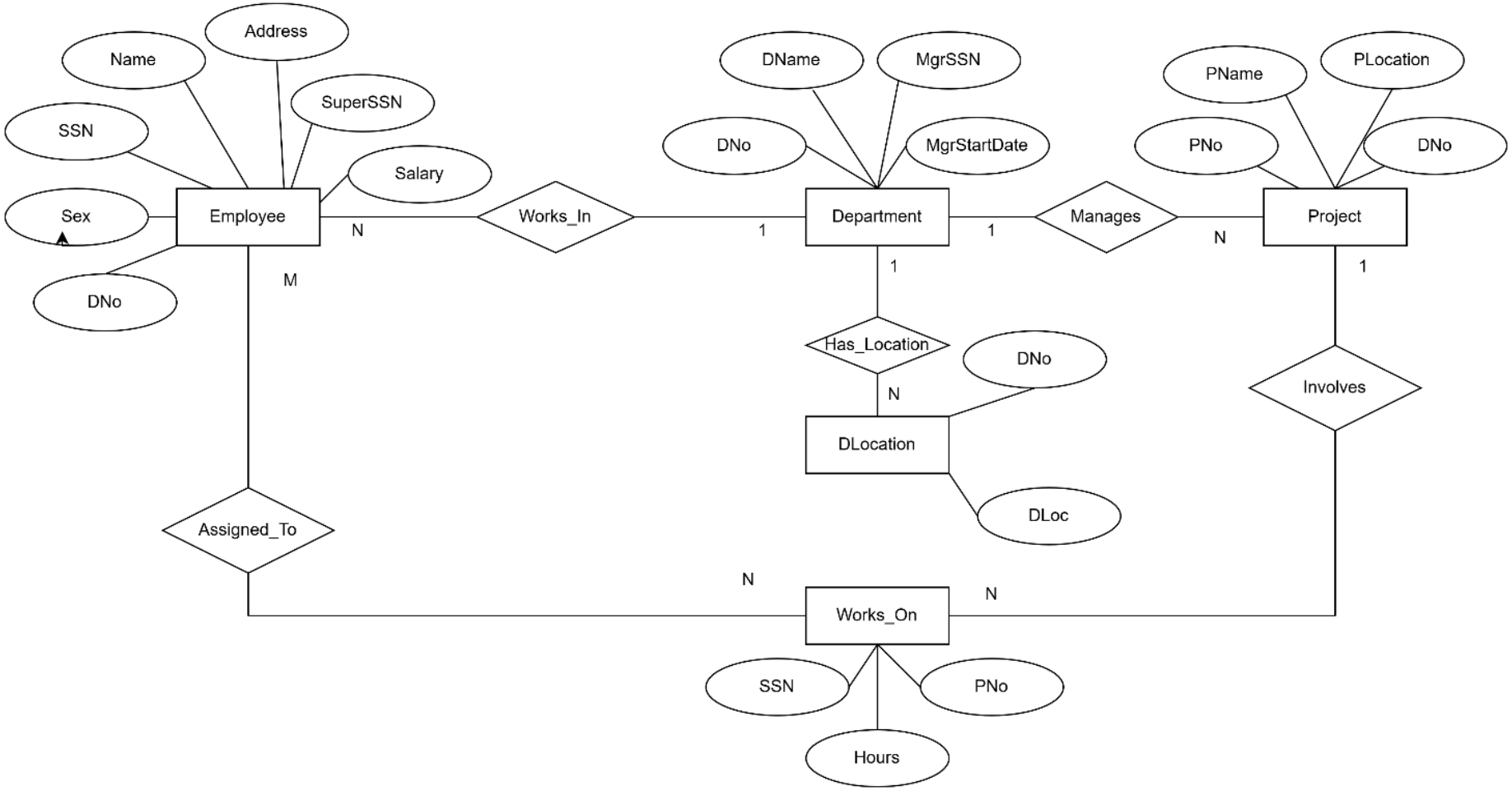
Entities involved: Employee – Project

Relationship type:

- **Many-to-Many (M : N)**

Explanation:
An employee can work on multiple projects, and a project can have multiple employees.
The attribute **Hours** indicates the number of hours an employee works on a project.

<u>Result:</u>


Address
Name
SuperSSN
SSN
Salary
Sex
Employee
DNo
N
M
Works_In
1
DName
MgrSSN
DNo
MgrStartDate
Department
1
1
Manages
N
PName
PLocation
PNo
DNo
Project
1
1
Has_Location
DNo
N
DLocation
DLoc
Involves
Assigned_To
N
N
Works_On
SSN
PNo
Hours

Experiment – 2

Aim

To design an Entity–Relationship (ER) diagram for a student academic management system and define the relationship types among entities.

Entities and Attributes

1. Student

Represents students enrolled in the institution.

Attributes:

- User_Name (Primary Key)
- Student_Name
- Address
- Phone_Number
- Gender

2. Semester_Section

Represents a specific section offered in a semester.

Attributes:

- SSection_ID (Primary Key)
- Semester
- Section

3. Class

Represents a class in which students are enrolled.

Attributes:

- SStudentID (Primary Key)
- User_Name

4. Subject

Represents subjects offered by the institution.

Attributes:

- Subject_Code (Primary Key)
- Title

- Credits
- Semester

5. IAMarks

Represents internal assessment marks of students.

Attributes:

- SSID
- Test1
- Test2
- Test3
- FinalIA

Relationships and Relationship Types

1. Enrolls

Entities involved: Student – Class

Relationship type:

- One-to-Many (1 : N)

Explanation:
One student can enroll in multiple classes, but each class enrollment is associated with only one student.

2. Contains

Entities involved: Semester_Section – Class

Relationship type:

- One-to-Many (1 : N)

Explanation:
A semester section can contain multiple classes, but each class belongs to only one semester section.

3. Has

Entities involved: Class – IAMarks

Relationship type:

- One-to-One (1 : 1)

Explanation:
Each class record has one set of internal assessment marks, and each IA marks record belongs to one class.

4. Assesses

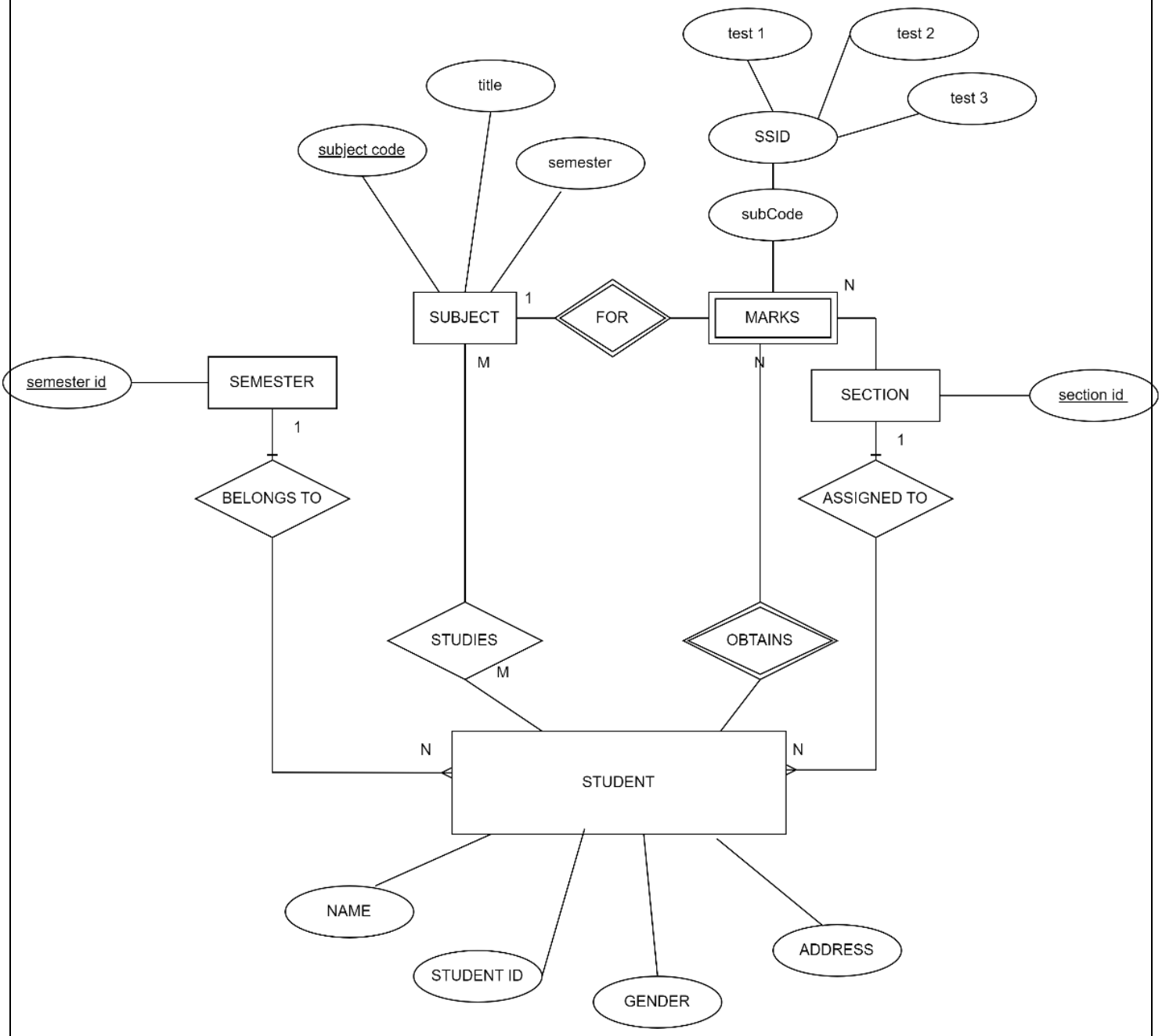
Entities involved: Subject – IAMarks

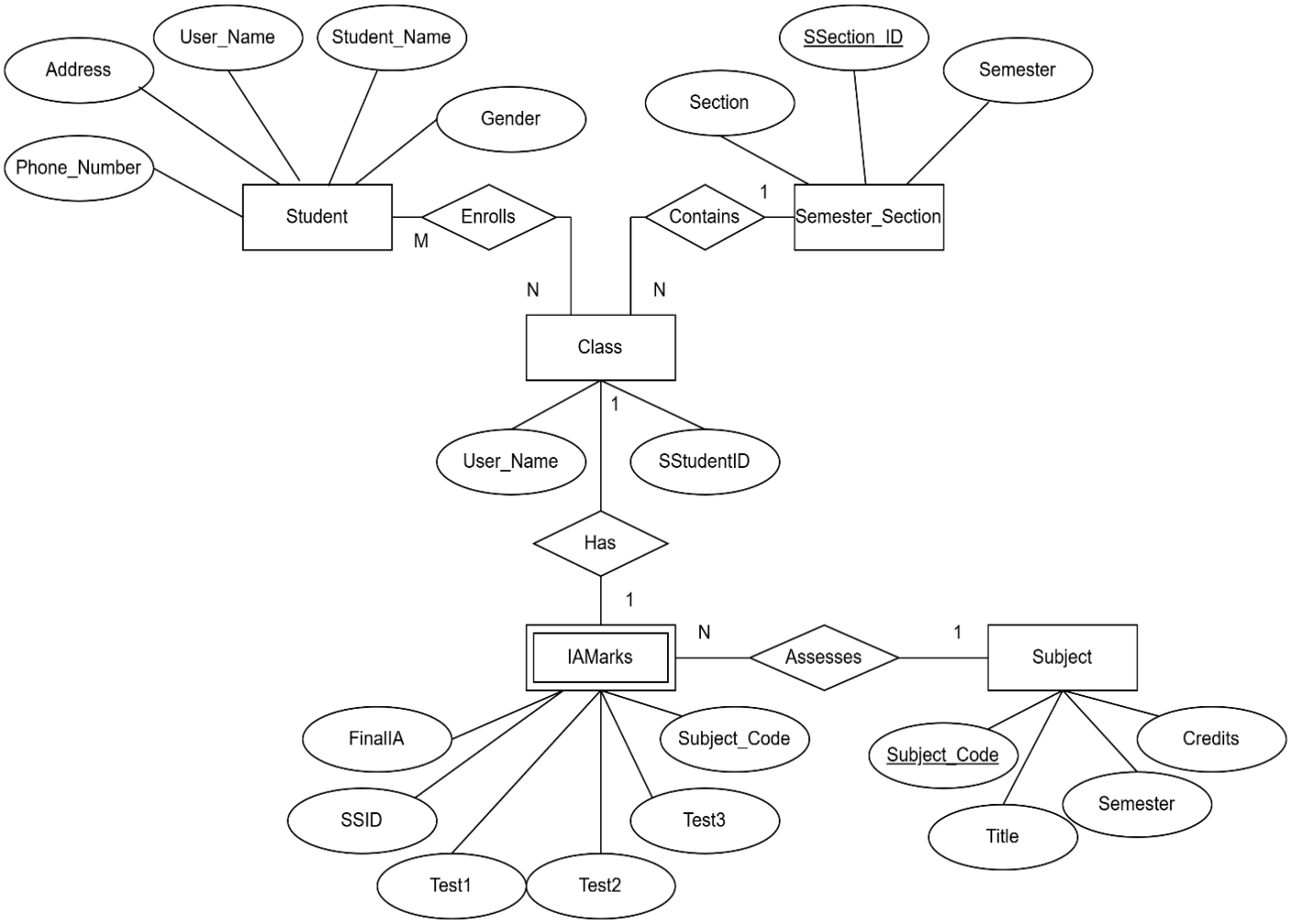
Relationship type:

- One-to-Many (1 : N)

Explanation:
A subject can assess many IA marks records for different students, but each IA marks record is related to only one subject.

Result:


test 1
test 2
test 3
title
SSID
subject code
semester
subCode
SUBJECT
1
FOR
MARKS
N
M
N
semester id
SEMESTER
SECTION
section id
1
1
BELONGS TO
ASSIGNED TO
STUDIES
OBTAINS
M
N
N
STUDENT
NAME
STUDENT ID
GENDER
ADDRESS


User_Name
Student_Name
Address
Gender
Phone_Number
Student
Enrolls
M
N
SSection_ID
Semester
Section
Contains
1
Semester_Section
N
Class
1
User_Name
SStudentID
Has
1
IAMarks
N
Assesses
1
Subject
FinalIA
Subject_Code
SSID
Test3
Test1
Test2
Subject_Code
Credits
Semester
Title

Experiment – 3

Create Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

2. Insert around 10 records in each of the tables.

3. List all the bills for the current date with the customer names and item numbers.

4. List the total Bill details with the quantity sold, price of the item and the final amount.

5. List the details of the customer who have bought a product which has a price>200.

6. Give a count of how many products have been bought by each customer

7. Give a list of products bought by a customer having cust_id as 5.

8. List the item details which are sold as of today.

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

10. Create a view which lists the daily sales date wise for the last one week

---

1-Create DATABASE

```
CREATE DATABASE salesdb;
        USE salesdb;
```

1.2 - Create the Tables with Integrity Constraints

```
CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
```

```
    cust_name VARCHAR(100) NOT NULL
);



CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    price INT CHECK (price > 0)
);

CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT CHECK (qty_sold > 0),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

2- Insert Around 10 Records in Each Table

```
INSERT INTO Customer VALUES
(1, 'Amit'),
 (2, 'Khushi'),
 (3, 'Ravi'),
 (4, 'Neha'),
 (5, 'Arjun'),
(6, 'Meera'),
(7, 'Sohan'),
```

(8, 'Priya'),

(9, 'Vikas'),

(10, 'Anita');

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

cust_id	cust_name
1	Amit
2	Khushi
3	Ravi
4	Neha
5	Arjun
6	Meera
7	Sohan
8	Priya
9	Vikas
10	Anita
NULL	NULL

customer 1

INSERT INTO Item VALUES

(101, 'Laptop', 55000),

(102, 'Mouse', 500),

(103, 'Keyboard', 1200),

(104, 'Monitor', 8000),

(105, 'Phone', 25000),

(106, 'Charger', 700),

(107, 'Headphones', 2200),

(108, 'Tablet', 18000),

(109, 'Camera', 30000),

(110, 'Speaker', 3500);

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

item_id	item_name	price
101	Laptop	55000
102	Mouse	500
103	Keyboard	1200
104	Monitor	8000
105	Phone	25000
106	Charger	700
107	Headphones	2200
108	Tablet	18000
109	Camera	30000
110	Speaker	3500
NULL	NULL	NULL

item 1

INSERT INTO Sale VALUES

(1001, CURDATE(), 1, 101, 1),

(1002, CURDATE(), 2, 102, 2),

(1003, CURDATE(), 3, 105, 1),

(1004, CURDATE(), 4, 107, 3),

(1005, CURDATE(), 5, 109, 1),

(1006, CURDATE(), 6, 110, 2),

(1007, CURDATE(), 7, 104, 1),

(1008, CURDATE(), 8, 103, 2),

(1009, CURDATE(), 9, 108, 1),

(1010, CURDATE(), 10, 106, 1);

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	qty_sold
1001	2026-02-18	1	101	1
1002	2026-02-18	2	102	2
1003	2026-02-18	3	105	1
1004	2026-02-18	4	107	3
1005	2026-02-18	5	109	1
1006	2026-02-18	6	110	2
1007	2026-02-18	7	104	1
1008	2026-02-18	8	103	2
1009	2026-02-18	9	108	1
1010	2026-02-18	10	106	1
NULL	NULL	NULL	NULL	NULL

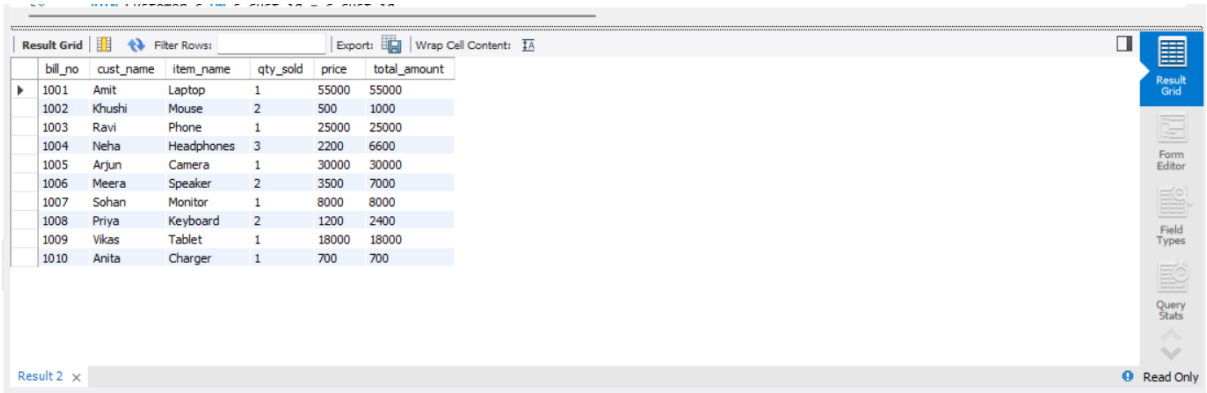
sale 1

3. List All Bills for Current Date with Customer Names and Item Numbers

```
SELECT s.bill_no, s.bill_date, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

4. List Total Bill Details (qty, price, final amount

```
SELECT s.bill_no, c.cust_name, i.item_name, s.qty_sold, i.price,
    (s.qty_sold * i.price) AS total_amount
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id;
```



bill_no	cust_name	item_name	qty_sold	price	total_amount
1001	Amit	Laptop	1	55000	55000
1002	Khushi	Mouse	2	500	1000
1003	Ravi	Phone	1	25000	25000
1004	Neha	Headphones	3	2200	6600
1005	Arjun	Camera	1	30000	30000
1006	Meera	Speaker	2	3500	7000
1007	Sohan	Monitor	1	8000	8000
1008	Priya	Keyboard	2	1200	2400
1009	Vikas	Tablet	1	18000	18000
1010	Anita	Charger	1	700	700

5.Customers Who Bought Products with Price > 200

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name
1	Amit
2	Khushi
8	Priya
7	Sohan
3	Ravi
10	Anita
4	Neha
9	Vikas
5	Arjun
6	Meera

Result 3 × Read Only

6. Count of Products Bought by Each Customer

SELECT c.cust_id, c.cust_name, COUNT(s.item_id) AS products_bought

FROM Sale s

JOIN Customer c ON s.cust_id = c.cust_id

GROUP BY c.cust_id, c.cust_name;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name	products_bought
1	Amit	1
2	Khushi	1
3	Ravi	1
4	Neha	1
5	Arjun	1
6	Meera	1
7	Sohan	1
8	Priya	1
9	Vikas	1
10	Anita	1

Result 4 × Read Only

7. Products Bought by Customer with cust_id = 5

SELECT i.item_id, i.item_name

FROM Sale s

JOIN Item i ON s.item_id = i.item_id

WHERE s.cust_id = 5;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_id	item_name
109	Camera

Result 5 × Read Only

8. Item Details Sold Today

```
SELECT DISTINCT i.item_id, i.item_name, i.price
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date = CURDATE()
```

9. Create a View for Bill Details

```
CREATE VIEW BillDetails AS
SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id,
    i.price, s.qty_sold, (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-18	1	101	55000	1	55000
1002	2026-02-18	2	102	500	2	1000
1003	2026-02-18	3	105	25000	1	25000
1004	2026-02-18	4	107	2200	3	6600
1005	2026-02-18	5	109	30000	1	30000
1006	2026-02-18	6	110	3500	2	7000
1007	2026-02-18	7	104	8000	1	8000
1008	2026-02-18	8	103	1200	2	2400
1009	2026-02-18	9	108	18000	1	18000
1010	2026-02-18	10	106	700	1	700

billdetails 1 ×

10. Create a View for Daily Sales (Last 1 Week)

```
CREATE VIEW WeeklySales AS
SELECT s.bill_date, SUM(s.qty_sold * i.price) AS daily_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
```

Experiment – 4/4.2

Consider the following table: Table: **CLASS**

ID	**Name**
1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- INSERT INTO class VALUES (5,'Rahul');
- COMMIT;
- UPDATE class SET name = 'Abhijeet' WHERE id= '5';
- SAVEPOINT A;
- INSERT INTO class VALUES (6, 'Chris');
- SAVEPOINT B;
- INSERT INTO class VALUES (7, 'Bravo');
- SAVEPOINT C
- SELECT * FROM class;
- ROLLBACK TO B;
- SELECT * FROM class;
- ROLLBACK TO A;

Experiment-4.2

Consider the following table: Table: **ACCOUNT_TXN**

Acc_ID	Holder_Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

- INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);
- SAVEPOINT S1;
- UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;
- UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;
- SAVEPOINT S2;
- DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;
- INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);
- SAVEPOINT S3;
- UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;
- SELECT * FROM ACCOUNT_TXN;
- ROLLBACK TO S2;
- SELECT * FROM ACCOUNT_TXN;
- INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);
- SAVEPOINT S4;
- UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);
- SELECT * FROM ACCOUNT_TXN;
- ROLLBACK TO S4;
- SELECT * FROM ACCOUNT_TXN;
- COMMIT;

1) Create Schema Database-

```
CREATE DATABASE Assignment4;

USE Assignment4;
```

2) Create Table

```
CREATE TABLE CLASS (
   ID INT PRIMARY KEY,
   Name VARCHAR(50)
);
```

3) Insert Initial Data

```
INSERT INTO CLASS VALUES (1, 'Bravo');

INSERT INTO CLASS VALUES (2, 'Alex');

INSERT INTO CLASS VALUES (4, 'Cheng');
```

4) Perform operations

```
INSERT INTO CLASS VALUES (5, 'Rahul');
```

COMMIT;

UPDATE CLASS SET Name = 'Abhijeet' WHERE ID = 5;

SAVEPOINT A;

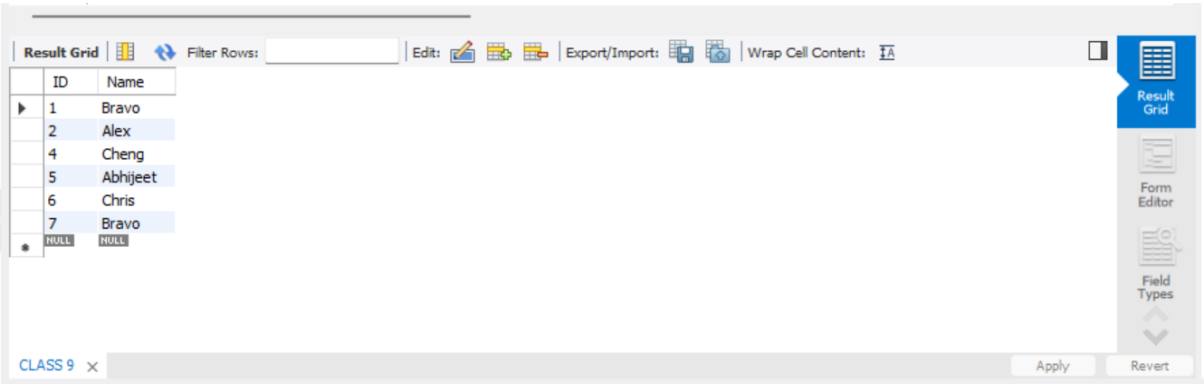
INSERT INTO CLASS VALUES (6, 'Chris');

SAVEPOINT B;

INSERT INTO CLASS VALUES (7, 'Bravo');

SAVEPOINT C;

SELECT * FROM CLASS;



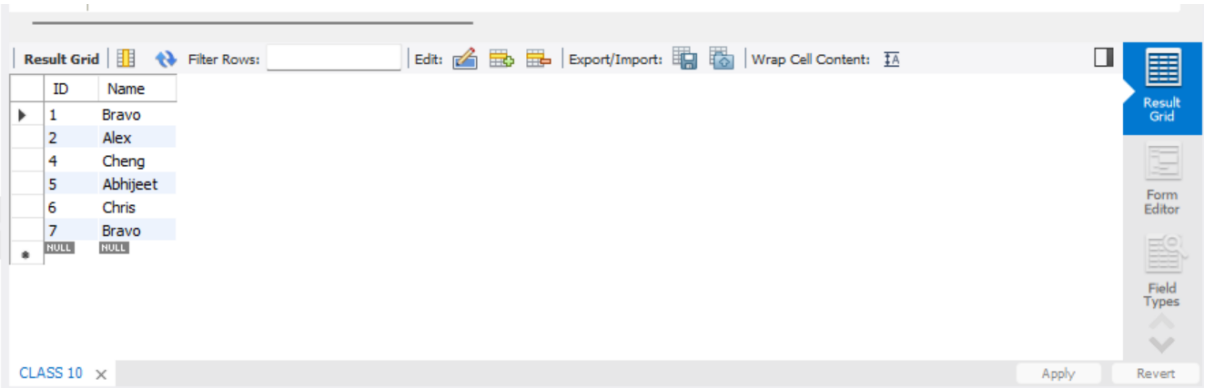
ID	Name
1	Bravo
2	Alex
4	Cheng
5	Abhijeet
6	Chris
7	Bravo
NULL	NULL

ROLLBACK TO B;

SELECT * FROM CLASS;

ROLLBACK TO A;

SELECT * FROM CLASS;



ID	Name
1	Bravo
2	Alex
4	Cheng
5	Abhijeet
6	Chris
7	Bravo
NULL	NULL

Experiment- 4.2

1) Create Table

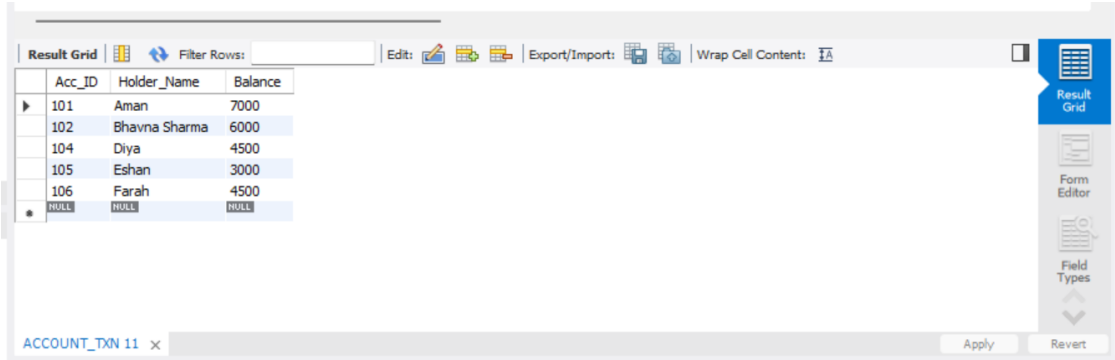
```
CREATE TABLE ACCOUNT_TXN (
    Acc_ID INT PRIMARY KEY,
    Holder_Name VARCHAR(50),
    Balance INT
);
```

2) Insert Initial Data

```
INSERT INTO ACCOUNT_TXN VALUES (101, 'Aman', 5000);
INSERT INTO ACCOUNT_TXN VALUES (102, 'Bhavna', 7000);
INSERT INTO ACCOUNT_TXN VALUES (103, 'Chirag', 6000);
```

3) Perform TCL Operations:-

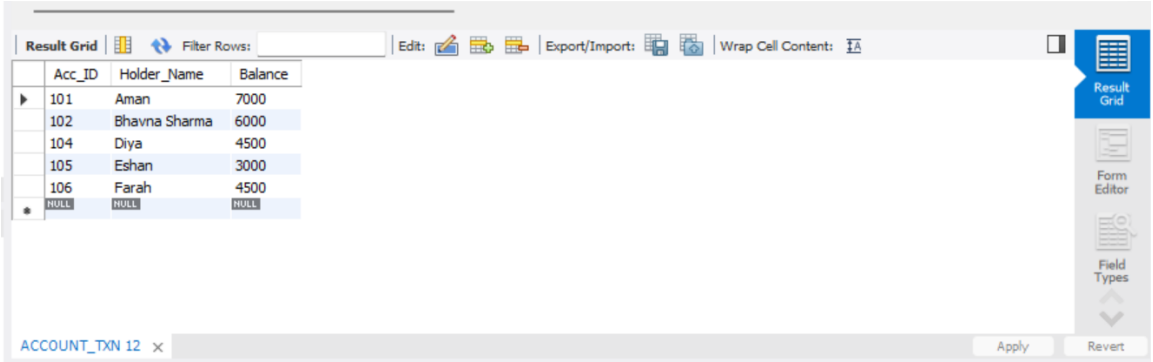
```
INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);
SAVEPOINT S1;

UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;
UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;
SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;
INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);
SAVEPOINT S3;

UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID
= 102;
SELECT * FROM ACCOUNT_TXN;
```



Acc_ID	Holder_Name	Balance
101	Aman	7000
102	Bhavna Sharma	6000
104	Diya	4500
105	Eshan	3000
106	Farah	4500
NULL	NULL	NULL

```
ROLLBACK TO S2;
```

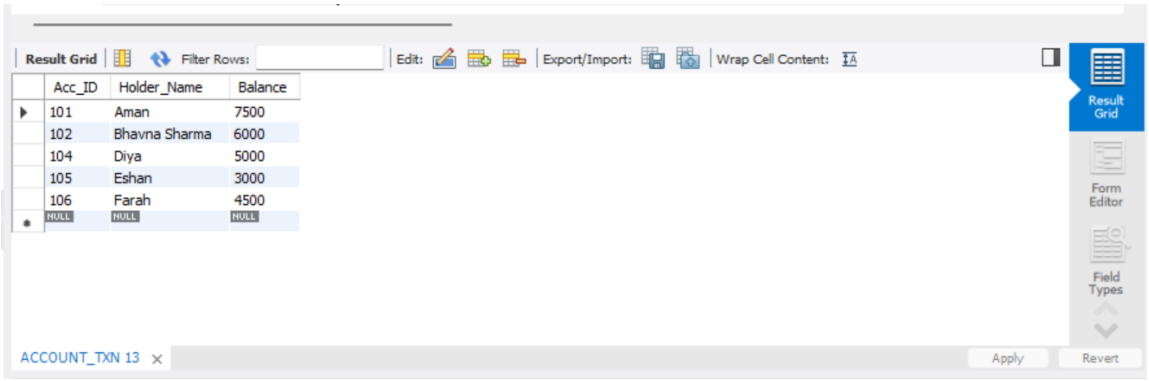
SELECT * FROM ACCOUNT_TXN;



Acc_ID	Holder_Name	Balance
101	Aman	7000
102	Bhavna Sharma	6000
104	Diya	4500
105	Eshan	3000
106	Farah	4500
NULL	NULL	NULL

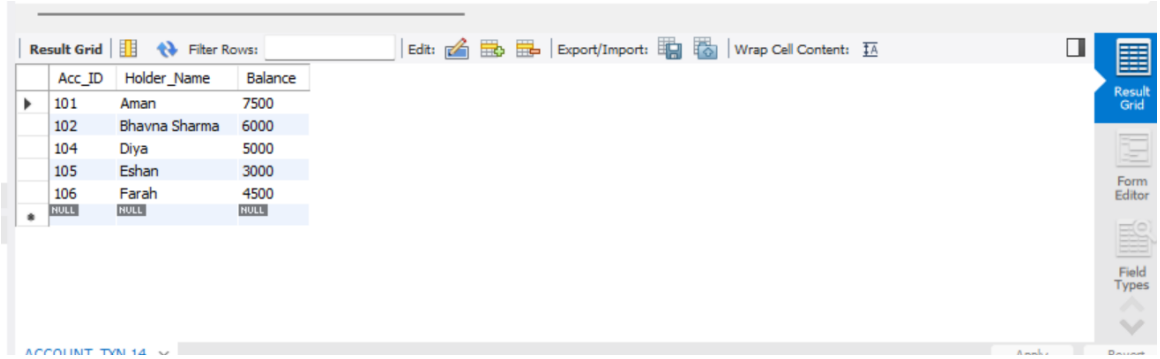
INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);
SAVEPOINT S4;

UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);
SELECT * FROM ACCOUNT_TXN;



Acc_ID	Holder_Name	Balance
101	Aman	7500
102	Bhavna Sharma	6000
104	Diya	5000
105	Eshan	3000
106	Farah	4500
NULL	NULL	NULL

ROLLBACK TO S4;
SELECT * FROM ACCOUNT_TXN;



Acc_ID	Holder_Name	Balance
101	Aman	7500
102	Bhavna Sharma	6000
104	Diya	5000
105	Eshan	3000
106	Farah	4500
NULL	NULL	NULL

COMMIT;

Experiment- 5

(Exercise on retrieving records from the table)

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)

1 Find out the employee id, names, salaries of all the employees
2 List out the employees who works under manager 100
3 Find the names of the employees who have a salary greater than or equal to 4800
4 List out the employees whose last name is 'AUSTIN'
5 Find the names of the employees who works in departments 60,70 and 80 (f) Display the unique Manager_Id.

1) Create Database and Table

```
CREATE DATABASE Assignment5;

USE Assignment5;

-- Create the EMPLOYEES table

CREATE TABLE EMPLOYEES (
    Employee_Id INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Email VARCHAR(100),
    Phone_Number VARCHAR(20),
    Hire_Date DATE,
    Job_Id VARCHAR(20),
    Salary INT,
    Commission_Act DECIMAL(5,2),
    Manager_Id INT,
    Department_Id INT
);
```

2) Insert Sample Data

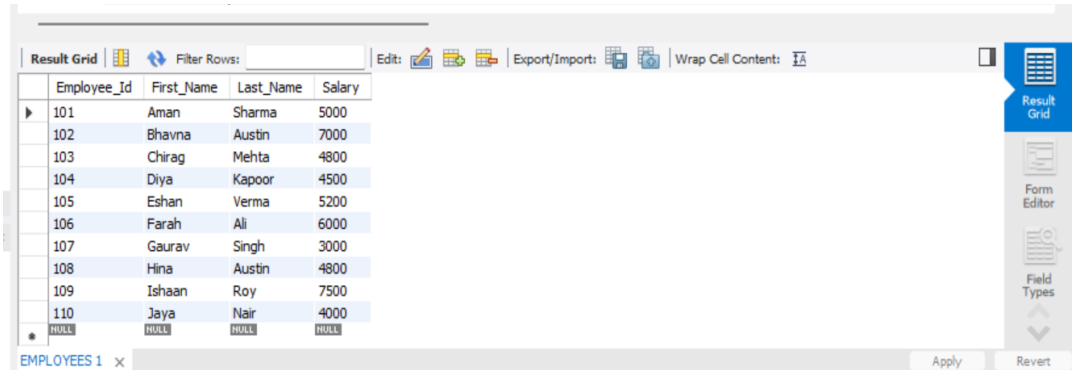
```
INSERT INTO EMPLOYEES VALUES
(101, 'Aman', 'Sharma', 'aman.sharma@company.com', '9876543210', '2020-01-15',
'DEV01', 5000, NULL, 100, 60),
```

(102, 'Bhavna', 'Austin', 'bhavna.austin@company.com', '9876543211', '2019-03-20', 'HR01', 7000, NULL, 101, 70),
(103, 'Chirag', 'Mehta', 'chirag.mehta@company.com', '9876543212', '2021-07-10', 'DEV02', 4800, NULL, 100, 80),
(104, 'Diya', 'Kapoor', 'diya.kapoor@company.com', '9876543213', '2022-05-05', 'FIN01', 4500, NULL, 102, 90),
(105, 'Eshan', 'Verma', 'eshan.verma@company.com', '9876543214', '2018-11-11', 'DEV03', 5200, NULL, 100, 60),
(106, 'Farah', 'Ali', 'farah.ali@company.com', '9876543215', '2020-09-09', 'HR02', 6000, NULL, 101, 70),
(107, 'Gaurav', 'Singh', 'gaurav.singh@company.com', '9876543216', '2017-02-02', 'DEV04', 3000, NULL, 102, 80),
(108, 'Hina', 'Austin', 'hina.austin@company.com', '9876543217', '2021-12-12', 'FIN02', 4800, NULL, 100, 60),
(109, 'Ishaan', 'Roy', 'ishaan.roy@company.com', '9876543218', '2019-06-06', 'DEV05', 7500, NULL, 103, 70),
(110, 'Jaya', 'Nair', 'jaya.nair@company.com', '9876543219', '2023-01-01', 'HR03', 4000, NULL, 101, 80);

3) QUERIES

(a) <u>Employee id, names, salaries of all employees</u>

```
SELECT Employee_Id, First_Name, Last_Name, Salary
FROM EMPLOYEES;
```



Employee_Id	First_Name	Last_Name	Salary
101	Aman	Sharma	5000
102	Bhavna	Austin	7000
103	Chirag	Mehta	4800
104	Diya	Kapoor	4500
105	Eshan	Verma	5200
106	Farah	Ali	6000
107	Gaurav	Singh	3000
108	Hina	Austin	4800
109	Ishaan	Roy	7500
110	Jaya	Nair	4000
NULL	NULL	NULL	NULL

(b) <u>Employees who work under manager 100</u>

```
SELECT Employee_Id, First_Name, Last_Name, Manager_Id
FROM EMPLOYEES
WHERE Manager_Id = 100;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Employee_Id	First_Name	Last_Name	Manager_Id
101	Aman	Sharma	100
103	Chirag	Mehta	100
105	Eshan	Verma	100
108	Hina	Austin	100
NULL	NULL	NULL	NULL

(c) <u>Employees with salary >= 4800</u>

```
SELECT First_Name, Last_Name, Salary
FROM EMPLOYEES
WHERE Salary >= 4800;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Last_Name	Salary
Aman	Sharma	5000
Bhavna	Austin	7000
Chirag	Mehta	4800
Eshan	Verma	5200
Farah	Ali	6000
Hina	Austin	4800
Ishaan	Roy	7500

EMPLOYEES 3

(d) <u>Employees whose last name is ‘AUSTIN’</u>

```
SELECT Employee_Id, First_Name, Last_Name
FROM EMPLOYEES
WHERE Last_Name = 'Austin';
```

Result Grid | Filter Rows: | Edit: | Export/Import:

Employee_Id	First_Name	Last_Name
102	Bhavna	Austin
108	Hina	Austin
NULL	NULL	NULL

(e) <u>Employees in departments 60, 70, 80</u>

```
SELECT First_Name, Last_Name, Department_Id
FROM EMPLOYEES
WHERE Department_Id IN (60, 70, 80);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Last_Name	Department_Id
Aman	Sharma	60
Bhavna	Austin	70
Chirag	Mehta	80
Eshan	Verma	60
Farah	Ali	70
Gaurav	Singh	80
Hina	Austin	60
Ishaan	Roy	70
Jaya	Nair	80

(f) <u>Unique Manager_Id</u>

```
SELECT DISTINCT Manager_Id
FROM EMPLOYEES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Manager_Id
100
101
102
103

Experiment- 6

(Exercise on updating records in table)

Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due)

1) Insert five records
2) Find the names of clients whose bal_due> 5000 .
3) Change the bal_due of ClientNO " C123" to Rs. 5100
4) change the name of Client_master to Client12.
5) Display the bal_due heading as "BALANCE"

1) Create Database Schema

```
CREATE DATABASE practical6;

USE practical6;
```

2) Create Table and Insert Records

```
CREATE TABLE Client_master (
    ClientNO VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    City VARCHAR(50),
    State VARCHAR(50),
    bal_due INT
);
```

```
INSERT INTO Client_master VALUES ('C101', 'Aman', '12 MG Road', 'Delhi', 'Delhi', 4500);

INSERT INTO Client_master VALUES ('C102', 'Bhavna', '45 Park Street', 'Kolkata', 'West Bengal', 7000);
```

INSERT INTO Client_master VALUES ('C123', 'Chirag', '78 Civil Lines', 'Lucknow', 'Uttar Pradesh', 5000);

INSERT INTO Client_master VALUES ('C104', 'Diya', '22 Connaught Place', 'Delhi', 'Delhi', 5200);

INSERT INTO Client_master VALUES ('C105', 'Eshan', '9 Brigade Road', 'Bangalore', 'Karnataka', 3000);

3) QUERIES

(a) Find the names of clients whose bal_due > 5000

SELECT Name, bal_due

FROM Client_master

WHERE bal_due > 5000;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Name	bal_due
Bhavna	7000
Diya	5200

(b) Change the bal_due of ClientNO “C123” to Rs. 5100

UPDATE Client_master

SET bal_due = 5100

WHERE ClientNO = 'C123';

(c) Change the name of table Client_master to Client12

ALTER TABLE Client_master

RENAME TO Client12;

(d) Display the bal_due heading as “BALANCE”

SELECT ClientNO, Name, bal_due AS BALANCE

FROM Client12;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ClientNO	Name	BALANCE
C101	Aman	4500
C102	Bhavna	7000
C104	Diya	5200
C105	Eshan	3000
C123	Chirag	5100
NULL	NULL	NULL

Experiment-7

Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

(a) Insert five records

(b) Give Increment of 25% salary for Mathematics Department . (c) Perform Rollback command (d)GiveIncrementof15%salaryforCommerce Department

(e) Perform commit command

Step 1: Create Database and Table

```
CREATE DATABASE Assignment7;

USE Assignment7;


CREATE TABLE Teacher (
    Name VARCHAR(50),
    DeptNo INT,
    Date_of_joining DATE,
    DeptName VARCHAR(50),
    Location VARCHAR(50),
    Salary INT
);
```

Step 2: Insert Five Records

```
INSERT INTO Teacher VALUES
('Aman', 10, '2020-01-15', 'Mathematics', 'Delhi', 40000),
('Bhavna', 20, '2019-03-20', 'Commerce', 'Kolkata', 35000),
('Chirag', 30, '2021-07-10', 'Science', 'Lucknow', 30000),
('Diya', 10, '2022-05-05', 'Mathematics', 'Delhi', 42000),
```

('Eshan', 20, '2018-11-11', 'Commerce', 'Bangalore', 37000);

Step 3: Queries

(b) Increment 25% salary for Mathematics Department:

```
UPDATE Teacher
SET Salary = Salary * 1.25
WHERE DeptName = 'Mathematics';
```

(c) Perform Rollback:

```
ROLLBACK;
```

(d) Increment 15% salary for Commerce Department:

```
UPDATE Teacher
SET Salary = Salary * 1.15
WHERE DeptName = 'Commerce';
```

(e) Perform Commit

```
COMMIT;
```

Experiment- 8

(Exercise on order by and group by clauses) Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)

(a) Insert five records

(b) Calculate total salesamount in each branch

(c) Calculate average salesamount in each branch (d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09

(e) Display the name and DOB of salesman in alphabetical order of the month.

Step 1: Create Database and Table

```
CREATE DATABASE Assignment8;
USE Assignment8;

CREATE TABLE Sales (
    SalesNo INT PRIMARY KEY,
    SalesName VARCHAR(50),
    Branch VARCHAR(50),
    SalesAmount INT,
    DOB DATE
);
```

Step 2: Insert Five Records

```
INSERT INTO Sales VALUES
(1, 'Aman', 'Delhi', 5000, '1990-12-21'),
(2, 'Bhavna', 'Kolkata', 7000, '1989-05-15'),
(3, 'Chirag', 'Delhi', 6000, '1992-12-09'),
```

(4, 'Diya', 'Mumbai', 8000, '1991-07-20'),

(5, 'Eshan', 'Kolkata', 4000, '1993-03-11');

Step 3: Queries

(b) Total sales per branch:

```
SELECT Branch, SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY Branch;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Branch	TotalSales
Delhi	11000
Kolkata	11000
Mumbai	8000

(c) Average sales per branch:

```
SELECT Branch, AVG(SalesAmount) AS AvgSales
FROM Sales
GROUP BY Branch;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Branch	AvgSales
Delhi	5500.0000
Kolkata	5500.0000
Mumbai	8000.0000

(d) Salesmen born in December (DOB as DD-Mon-YY):

```
SELECT SalesName, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB
FROM Sales
```

WHERE MONTH(DOB) = 12;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SalesName	DOB
Aman	21-Dec-90
Chirag	09-Dec-92

(e) Salesmen in alphabetical order of month:

SELECT SalesName, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB

FROM Sales

ORDER BY MONTH(DOB);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SalesName	DOB
Eshan	11-Mar-93
Bhavna	15-May-89
Diya	20-Jul-91
Aman	21-Dec-90
Chirag	09-Dec-92

Experiment -9

Consider the following tables namely "DEPARTMENTS" and "EMPLOYEES" Their schema are as follows:-

Departments (dept no , dept name , dept_location); Employees (emp_id , emp_name , emp_salary,dept_no);

a) Develop a query to grant all privileges of employees table into departments table

b) Develop a query to grant some privileges of employees table into departments table

c) Develop a query to revoke all privileges of employees table from departments table

d) Develop a query to revoke some privileges of employees table from departments table

e) Write a query to implement the save point.

Step 1: Create Database and Tables

```
CREATE DATABASE practical9;

USE practical9;


CREATE TABLE Departments (

   DeptNo INT PRIMARY KEY,

   DeptName VARCHAR(50),

   Dept_Location VARCHAR(50)

);


CREATE TABLE Employees (

   Emp_Id INT PRIMARY KEY,

   Emp_Name VARCHAR(50),

   Emp_Salary INT,

   Dept_No INT

);
```

Step 2: Queries

(a) Grant all privileges:

GRANT ALL PRIVILEGES ON Employees TO Departments;

(b) Grant some privileges:

GRANT SELECT, INSERT ON Employees TO Departments;

(c) Revoke all privileges:

REVOKE ALL PRIVILEGES ON Employees FROM Departments;

(d) Revoke some privileges:

REVOKE INSERT ON Employees FROM Departments;

(e) Implement savepoint:

SAVEPOINT A;

Experiment -10

Using the tables "DEPARTMENTS" and "EMPLOYEES" perform the following queries

a) Display the employee details, departments that the departments are same in both the emp and dept.

b) Display the employee name and Department name by implementing a left outer join.

c) Display the employee name and Department name by implementing a right outer join.

d) Display the details of those who draw the salary greater than the average salary

Step 1: Create Database and Tables

```
CREATE DATABASE practical9;

USE practical9;


CREATE TABLE Departments (

   DeptNo INT PRIMARY KEY,

   DeptName VARCHAR(50),

   Dept_Location VARCHAR(50)

);


CREATE TABLE Employees (

   Emp_Id INT PRIMARY KEY,

   Emp_Name VARCHAR(50),

   Emp_Salary INT,

   Dept_No INT

);
```

Step 2: Records

```
INSERT INTO Departments VALUES
(10, 'Mathematics', 'Delhi'),
(20, 'Commerce', 'Kolkata'),
(30, 'Science', 'Lucknow'),
(40, 'Finance', 'Mumbai');

INSERT INTO Employees VALUES
(101, 'Aman', 40000, 10),
(102, 'Bhavna', 35000, 20),
(103, 'Chirag', 30000, 30),
(104, 'Diya', 42000, 10),
(105, 'Eshan', 37000, 20),
(106, 'Farah', 28000, NULL),   -- Employee without department
(107, 'Gaurav', 50000, 40),
(108, 'Hina', 32000, 30);
```

Step 3: Queries

(a) Employee details where departments match:

```
SELECT e.Emp_Id, e.Emp_Name, e.Emp_Salary, d.DeptName, d.Dept_Location
FROM Employees e
JOIN Departments d ON e.Dept_No = d.DeptNo;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Emp_Id	Emp_Name	Emp_Salary	DeptName	Dept_Location
101	Aman	40000	Mathematics	Delhi
102	Bhavna	35000	Commerce	Kolkata
103	Chirag	30000	Science	Lucknow
104	Diya	42000	Mathematics	Delhi
105	Eshan	37000	Commerce	Kolkata
107	Gaurav	50000	Finance	Mumbai
108	Hina	32000	Science	Lucknow

(b) Left outer join:

SELECT e.Emp_Name, d.DeptName

FROM Employees e

LEFT JOIN Departments d ON e.Dept_No = d.DeptNo;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Emp_Name	DeptName
Aman	Mathematics
Bhavna	Commerce
Chirag	Science
Diya	Mathematics
Eshan	Commerce
Farah	NULL
Gaurav	Finance
Hina	Science

<u>(c) Right outer join:</u>

SELECT e.Emp_Name, d.DeptName

FROM Employees e

RIGHT JOIN Departments d ON e.Dept_No = d.DeptNo;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Emp_Name	DeptName
Diya	Mathematics
Aman	Mathematics
Eshan	Commerce
Bhavna	Commerce
Hina	Science
Chirag	Science
Gaurav	Finance

<u>(d) Employees with salary greater than average:</u>

SELECT Emp_Name, Emp_Salary

FROM Employees

WHERE Emp_Salary > (SELECT AVG(Emp_Salary) FROM Employees);

Result Grid | Filter Rows: | Export: | Wrap Cell Content

Emp_Name	Emp_Salary
Aman	40000
Diya	42000
Eshan	37000
Gaurav	50000